

■ LEVEL 23 • BFS

Pattern Recognition • Problem Practice • Optimal C++ • Complexity

Pattern Trigger	Level-by-level exploration or shortest path in an unweighted graph. Use a queue.
Reference Examples	Binary Tree Level Order • Number of Islands • Rotting Oranges • Shortest Path in Binary Matrix • Word Ladder
Problems	6 focused problems in this level
Code Standard	C++ • visible nested indentation • syntax-colored keywords/functions/comments/strings

Reference framework: the user's **DSA Approach Handbook**. The handbook defines the trigger, why, key thought, complexity, and representative practice problems; this continuation turns each level into a problem-by-problem practice book.

LEVEL 23 • EXPANDED PROBLEM INDEX

The original 6 core problems are retained. Four additional problems are added below, giving 10 total problems in this level.

#	Problem	Pattern	LeetCode
1	Binary Tree Level Order Traversal	BFS + Queue	#102
2	Rotting Oranges	Multi-source BFS	#994
3	Shortest Path in Binary Matrix	Grid BFS	#1091
4	Word Ladder	BFS + Word Set	#127
5	Open the Lock	BFS State Space	#752
6	01 Matrix	Multi-source BFS	#542
7	Binary Tree Level Order Traversal	BFS	102
8	Rotting Oranges	Multi-Source BFS	994
9	Shortest Path in Binary Matrix	BFS Shortest Path	1091
10	Word Ladder	BFS + Implicit Graph	127

Extension rule: Problems 7–10 are additional practice chosen to strengthen the same pattern. They are clearly separated from the reference handbook's original representative set.

PROBLEM 1 Binary Tree Level Order Traversal

Pattern: BFS + Queue

LeetCode: #102

■ Input	binary tree
■ Output	values level by level
■ Constraints	binary tree
■ Main Idea / Pattern	Queue guarantees nodes are processed in distance/level order.
■■ Time Complexity	O(n)
■ Space Complexity	O(w)

■ Optimal C++ Code

```
vector<vector<int>> levelOrder(TreeNode* root) {  
    if (!root) return {};  
    vector<vector<int>> ans;  
    queue<TreeNode*> q;  
    q.push(root);  
    while (!q.empty()) {  
        int sz = q.size();  
        vector<int> level;  
        while (sz-->0) {  
            auto node = q.front();  
            q.pop();  
            level.push_back(node->val);  
            if (node->left) q.push(node->left);  
            if (node->right) q.push(node->right);  
        }  
        ans.push_back(level);  
    }  
    return ans;  
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 2 Rotting Oranges

Pattern: Multi-source BFS

LeetCode: #994

■ Input	grid with fresh and rotten oranges
■ Output	minutes to rot all fresh oranges
■ Constraints	four-direction movement
■ Main Idea / Pattern	All initially rotten cells start at distance zero, then BFS expands one minute at a time.
■ Time Complexity	O(mn)
■ Space Complexity	O(mn)

■ Optimal C++ Code

```
int orangesRotting(vector<vector<int>>& grid) {
    int m = grid.size(), n = grid[0].size();
    queue<pair<int, int>> q;
    int fresh = 0;

    for (int r = 0; r < m; r++)
        for (int c = 0; c < n; c++) {
            if (grid[r][c] == 2) q.push({r, c});
            if (grid[r][c] == 1) fresh++;
        }

    int minutes = 0;
    int dr[4] = {-1, 1, 0, 0};
    int dc[4] = {0, 0, -1, 1};

    while (!q.empty() && fresh) {
        int sz = q.size();
        while (sz--) {
            auto [r, c] = q.front();
            q.pop();

            for (int d = 0; d < 4; d++) {
                int nr = r + dr[d], nc = c + dc[d];

                if (nr >= 0 && nr < m && nc >= 0 && nc < n &&
                    grid[nr][nc] == 1) {
                    grid[nr][nc] = 2;
                    fresh--;
                    q.push({nr, nc});
                }
            }
        }
        minutes++;
    }

    return fresh ? -1 : minutes;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 3 Shortest Path in Binary Matrix

Pattern: Grid BFS

LeetCode: #1091

■ Input	binary square grid
■ Output	shortest clear path length
■ Constraints	8-direction movement
■ Main Idea / Pattern	BFS from the source. The first time a cell is reached gives its shortest distance.
■ Time Complexity	$O(n^2)$
■ Space Complexity	$O(n^2)$

■ Optimal C++ Code

```
int shortestPathBinaryMatrix(vector<vector<int>>& grid) {
    int n = grid.size();
    if (grid[0][0] || grid[n - 1][n - 1]) return -1;

    queue<pair<int, int>> q;
    q.push({0, 0});
    grid[0][0] = 1;

    int dr[8] = {-1, -1, -1, 0, 0, 1, 1, 1};
    int dc[8] = {-1, 0, 1, -1, 1, -1, 0, 1};

    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();

        if (r == n - 1 && c == n - 1)
            return grid[r][c];

        for (int d = 0; d < 8; d++) {
            int nr = r + dr[d], nc = c + dc[d];

            if (nr >= 0 && nr < n && nc >= 0 && nc < n &&
                grid[nr][nc] == 0) {
                grid[nr][nc] = grid[r][c] + 1;
                q.push({nr, nc});
            }
        }
    }

    return -1;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 4 Word Ladder

Pattern: BFS + Word Set

LeetCode: #127

■ Input	begin word, end word, dictionary
■ Output	minimum number of word transformations
■ Constraints	one-letter changes
■ Main Idea / Pattern	Each valid word is a graph node. BFS explores transformations by number of steps.
■ Time Complexity	$O(N \cdot L^2)$ typical
■ Space Complexity	$O(N)$

■ Optimal C++ Code

```
int ladderLength(string beginWord, string endWord,
                vector<string>& wordList) {
    unordered_set<string> dict(wordList.begin(), wordList.end());
    if (!dict.count(endWord)) return 0;

    queue<string> q;
    q.push(beginWord);
    int steps = 1;

    while (!q.empty()) {
        int sz = q.size();
        while (sz-- > 0) {
            string word = q.front();
            q.pop();

            if (word == endWord) return steps;

            for (int i = 0; i < word.size(); i++) {
                char old = word[i];

                for (char c = 'a'; c <= 'z'; c++) {
                    word[i] = c;

                    if (dict.count(word)) {
                        dict.erase(word);
                        q.push(word);
                    }
                }

                word[i] = old;
            }
        }
        steps++;
    }

    return 0;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 5 Open the Lock

Pattern: BFS State Space

LeetCode: #752

■ Input	deadends and target
■ Output	minimum turns
■ Constraints	four-digit state
■ Main Idea / Pattern	Treat every lock configuration as a node and each wheel turn as an edge.
■ Time Complexity	$O(10^4)$
■ Space Complexity	$O(10^4)$

■ Optimal C++ Code

```
int openLock(vector<string>& deadends, string target) {
    unordered_set<string> dead(deadends.begin(), deadends.end());
    if (dead.count("0000")) return -1;

    queue<string> q;
    unordered_set<string> seen;
    q.push("0000");
    seen.insert("0000");

    int steps = 0;
    while (!q.empty()) {
        int sz = q.size();
        while (sz--) {
            string cur = q.front();
            q.pop();

            if (cur == target) return steps;

            for (int i = 0; i < 4; i++) {
                for (int d : {-1, 1}) {
                    string nxt = cur;
                    nxt[i] = char((nxt[i] - '0' + d + 10) % 10 + '0');

                    if (!dead.count(nxt) && !seen.count(nxt)) {
                        seen.insert(nxt);
                        q.push(nxt);
                    }
                }
            }
        }
        steps++;
    }
    return -1;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 6 01 Matrix

Pattern: Multi-source BFS

LeetCode: #542

■ Input	binary matrix
■ Output	distance of each cell to nearest zero
■ Constraints	at least one zero exists
■ Main Idea / Pattern	Start BFS from all zeros simultaneously. Distances grow outward layer by layer.
■■ Time Complexity	O(mn)
■ Space Complexity	O(mn)

■ Optimal C++ Code

```
vector<vector<int>> updateMatrix(vector<vector<int>>& mat) {
    int m = mat.size(), n = mat[0].size();
    queue<pair<int, int>> q;

    for (int r = 0; r < m; r++)
        for (int c = 0; c < n; c++) {
            if (mat[r][c] == 0) q.push({r, c});
            else mat[r][c] = -1;
        }

    int dr[4] = {-1, 1, 0, 0};
    int dc[4] = {0, 0, -1, 1};

    while (!q.empty()) {
        auto [r, c] = q.front();
        q.pop();

        for (int d = 0; d < 4; d++) {
            int nr = r + dr[d], nc = c + dc[d];

            if (nr >= 0 && nr < m && nc >= 0 && nc < n &&
                mat[nr][nc] == -1) {
                mat[nr][nc] = mat[r][c] + 1;
                q.push({nr, nc});
            }
        }
    }

    return mat;
}
```

Practice note: Re-solve without looking. Then change one condition (input order, duplicates, edge case, or constraint) and explain what part of the pattern changes.

PROBLEM 7 Binary Tree Level Order Traversal

Pattern: BFS

LeetCode: 102

Input	root
Output	values level by level
Constraints	0 <= nodes <= 2000
Main Idea / Pattern	Queue nodes and process one queue layer at a time.
Time Complexity	O(n)
Space Complexity	O(w)

Optimal C++ Code

```
class Solution {
public:
    vector<vector<int>> levelOrder(TreeNode* root) {
        if (!root) return {};
        vector<vector<int>> ans;
        queue<TreeNode*> q;
        q.push(root);
        while (!q.empty()) {
            int sz = q.size();
            vector<int> level;
            while (sz-- > 0) {
                TreeNode* cur = q.front();
                q.pop();
                level.push_back(cur->val);
                if (cur->left) q.push(cur->left);
                if (cur->right) q.push(cur->right);
            }
            ans.push_back(level);
        }
        return ans;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 8 Rotting Oranges

Pattern: Multi-Source BFS

LeetCode: 994

Input	grid
Output	minutes until all possible oranges rot
Constraints	rows, cols <= 10
Main Idea / Pattern	Start BFS from every rotten orange simultaneously; each layer represents one minute.
Time Complexity	O(mn)
Space Complexity	O(mn)

Optimal C++ Code

```
class Solution {
public:
    int orangesRotting(vector<vector<int>>& grid) {
        int m = grid.size();
        int n = grid[0].size();
        queue<pair<int, int>> q;
        int fresh = 0;

        for (int r = 0; r < m; r++) {
            for (int c = 0; c < n; c++) {
                if (grid[r][c] == 2) q.push({r, c});
                if (grid[r][c] == 1) fresh++;
            }
        }

        int minutes = 0;
        int dr[] = {-1, 1, 0, 0};
        int dc[] = {0, 0, -1, 1};

        while (!q.empty() && fresh) {
            int sz = q.size();
            minutes++;

            while (sz-- > 0) {
                auto [r, c] = q.front();
                q.pop();

                for (int d = 0; d < 4; d++) {
                    int nr = r + dr[d];
                    int nc = c + dc[d];

                    if (nr >= 0 && nr < m &&
                        nc >= 0 && nc < n &&
                        grid[nr][nc] == 1) {
                        grid[nr][nc] = 2;
                        fresh--;
                        q.push({nr, nc});
                    }
                }
            }
        }

        return fresh ? -1 : minutes;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 9 Shortest Path in Binary Matrix

Pattern: BFS Shortest Path

LeetCode: 1091

Input	grid
Output	shortest clear path length
Constraints	n <= 100; cells are 0 or 1
Main Idea / Pattern	Because every move costs one edge, BFS reaches the target with the minimum number of steps.
Time Complexity	O(n^2)
Space Complexity	O(n^2)

Optimal C++ Code

```
class Solution {
public:
    int shortestPathBinaryMatrix(vector<vector<int>>& grid) {
        int n = grid.size();
        if (grid[0][0] || grid[n - 1][n - 1]) return -1;
        queue<pair<int, int>> q;
        q.push({0, 0});
        grid[0][0] = 1;

        int dr[] = {-1, -1, -1, 0, 0, 1, 1, 1};
        int dc[] = {-1, 0, 1, -1, 1, -1, 0, 1};
        int dist = 1;

        while (!q.empty()) {
            int sz = q.size();
            while (sz-- > 0) {
                auto [r, c] = q.front();
                q.pop();

                if (r == n - 1 && c == n - 1) return dist;

                for (int d = 0; d < 8; d++) {
                    int nr = r + dr[d];
                    int nc = c + dc[d];

                    if (nr >= 0 && nr < n &&
                        nc >= 0 && nc < n &&
                        grid[nr][nc] == 0) {
                        grid[nr][nc] = 1;
                        q.push({nr, nc});
                    }
                }
            }
            dist++;
        }
        return -1;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.

PROBLEM 10 Word Ladder

Pattern: BFS + Implicit Graph

LeetCode: 127

Input	beginWord, endWord, wordList
Output	minimum transformation sequence length
Constraints	word length <= 10; wordList size <= 5000
Main Idea / Pattern	Each one-letter mutation is an edge; BFS finds the shortest transformation sequence.
Time Complexity	$O(N * L^2)$ typical
Space Complexity	$O(N)$

Optimal C++ Code

```
class Solution {
public:
    int ladderLength(string beginWord, string endWord,
                    vector<string>& wordList) {
        unordered_set<string> words(wordList.begin(), wordList.end());
        if (!words.count(endWord)) return 0;
        queue<string> q;
        q.push(beginWord);
        int steps = 1;
        while (!q.empty()) {
            int sz = q.size();
            while (sz-- > 0) {
                string cur = q.front();
                q.pop();
                if (cur == endWord) return steps;
                for (int i = 0; i < cur.size(); i++) {
                    char old = cur[i];
                    for (char c = 'a'; c <= 'z'; c++) {
                        cur[i] = c;
                        if (words.count(cur)) {
                            words.erase(cur);
                            q.push(cur);
                        }
                    }
                    cur[i] = old;
                }
            }
            steps++;
        }
        return 0;
    }
};
```

Practice: Re-solve without looking, then change one constraint and explain which part of the pattern changes.